

SAE Intégrative 2.04

Partie Réseau & Téléphonie — Dossier de configuration
Groupe 6 · Réseau LAN 10.252.6.0/24 · Découpage en 4 sous-réseaux /26

Plan d'adressage

Le réseau /24 est découpé en quatre sous-réseaux /26 (62 hôtes utiles chacun), un par VLAN. Les dix premières adresses de chaque plage sont réservées aux adresses statiques.

VLAN	Nom	Sous-réseau	Passerelle (routeur)	Plage DHCP	Ports switch
100	Voix	10.252.6.0/26	10.252.6.1	.11 – .62	Gi1/0/2-3
200	Users	10.252.6.64/26	10.252.6.65	.75 – .126	Gi1/0/4-5
300	Serveur	10.252.6.128/26	10.252.6.129	.139 – .190	Gi1/0/6-7
400	Administrateur	10.252.6.192/26	10.252.6.193	.203 – .254	Gi1/0/8-9

Le serveur Debian a une IP **statique** : 10.252.6.130 (VLAN 300), passerelle 10.252.6.129, DNS 10.252.4.42.

Câblage physique

Routeur **Gi0/0** ↔ Switch **Gi1/0/1** (lien trunk, transporte les 4 VLANs) · Routeur **Gi0/1** ↔ réseau UHA (accès Internet, IP obtenue en DHCP) · les postes et le serveur se branchent sur les ports d'accès du switch.

I Gestion des VLAN

But : Séparer le réseau en 4 VLANs (Voix, Users, Serveur, Admin), faire communiquer ces VLANs entre eux via le routeur (router-on-a-stick), distribuer les adresses en DHCP, et empêcher le VLAN 200 de pinger le VLAN 400 tout en autorisant l'inverse.

1. Création des VLANs et des ports (switch Catalyst 3850)

```
configure terminal
vlan 100
name Voix
vlan 200
name Users
vlan 300
name Serveur
vlan 400
name Administrateur
exit

! 2 ports d'accès par VLAN (exemple VLAN 100)
interface Gi1/0/2
switchport mode access
switchport access vlan 100
interface Gi1/0/3
switchport mode access
switchport access vlan 100
! ... idem Gi1/0/4-5 (vlan 200), Gi1/0/6-7 (vlan 300), Gi1/0/8-9 (vlan 400)
exit

! Lien trunk vers le routeur
interface Gi1/0/1
switchport mode trunk
exit
```

Le port trunk transporte simultanément les 4 VLANs vers le routeur grâce au marquage 802.1Q. Les ports d'accès appartiennent chacun à un seul VLAN.

2. Routage inter-VLAN + DHCP (routeur Cisco 1921)

Le routeur crée une sous-interface par VLAN sur son port Gi0/0. Chaque sous-interface est la passerelle de son VLAN et distribue les adresses en DHCP.

```

configure terminal
interface GigabitEthernet0/0
no shutdown
interface Gi0/0.100
encapsulation dot1Q 100
ip address 10.252.6.1 255.255.255.192
interface Gi0/0.200
encapsulation dot1Q 200
ip address 10.252.6.65 255.255.255.192
interface Gi0/0.300
encapsulation dot1Q 300
ip address 10.252.6.129 255.255.255.192
interface Gi0/0.400
encapsulation dot1Q 400
ip address 10.252.6.193 255.255.255.192
exit

! Adresses statiques reservees (10 premieres par plage)
ip dhcp excluded-address 10.252.6.1 10.252.6.10
ip dhcp excluded-address 10.252.6.65 10.252.6.74
ip dhcp excluded-address 10.252.6.129 10.252.6.138
ip dhcp excluded-address 10.252.6.193 10.252.6.202

! Pool DHCP par VLAN (exemple VLAN 100)
ip dhcp pool VLAN100
network 10.252.6.0 255.255.255.192
default-router 10.252.6.1
dns-server 10.252.4.42
! ... idem VLAN200 / VLAN300 / VLAN400

```

3. ACL : bloquer ping VLAN 200 → VLAN 400 (autoriser l'inverse)

```

ip access-list extended BLOCK_PING_200_TO_400
deny icmp 10.252.6.64 0.0.0.63 10.252.6.192 0.0.0.63
permit ip any any
exit
interface Gi0/0.200
ip access-group BLOCK_PING_200_TO_400 in

```

L'ACL est appliquée en entrée sur la sous-interface du VLAN 200 : elle filtre le trafic qui part du VLAN 200. On bloque uniquement l'ICMP vers le VLAN 400 ; le retour (400 → 200) n'est pas filtré, donc le ping fonctionne dans ce sens.

Validation

- Un poste branché dans un VLAN reçoit bien une adresse de la bonne plage en DHCP (ex. 10.252.6.11 en VLAN 100, 10.252.6.75 en VLAN 200).
- Un poste du VLAN 100 ping un poste du VLAN 200 (TTL = 63 → passe par le routeur).
- Commandes de vérif : `show vlan brief`, `show interfaces trunk`, `show ip dhcp binding`.

II Protection du commutateur

But : Protéger le switch des boucles réseau et bloquer automatiquement un port si du trafic Spanning-Tree (BPDU) y est détecté.

Configuration (switch)

```
configure terminal
interface range Gi1/0/2 - 9
spanning-tree bpduguard enable
exit
```

Le **Spanning-Tree (STP)** est actif par défaut et empêche les boucles entre switches. Le **BPDU Guard** ajouté sur les ports d'accès complète cette protection : si un port reçoit un paquet BPDU (signe qu'un switch non autorisé a été branché), le port passe immédiatement en état **err-disabled** et coupe le trafic. C'est exactement ce que demande le sujet : « blocage des ports si du trafic lié au spanning-tree est détecté ».

Validation

show spanning-tree summary indique BPDU Guard activé ; show spanning-tree detail | include BPDU montre les compteurs (BPDU envoyés par le switch, 0 reçu sur les ports d'accès).

III Port mirroring (SPAN)

But : Recopier le trafic réseau vers des ports d'analyse, afin de pouvoir le capturer (Wireshark / tcpdump) pour le débogage, sans perturber le réseau. Deux miroirs sont demandés : le port vers le routeur, et le VLAN 100.

Configuration (switch)

```
configure terminal
! Session 1 : trafic entree/sortie du port vers le routeur
monitor session 1 source interface Gi1/0/1 both
monitor session 1 destination interface Gi1/0/10

! Session 2 : tout le trafic du VLAN 100
monitor session 2 source vlan 100
monitor session 2 destination interface Gi1/0/11
```

Un PC d'analyse branché sur Gi1/0/10 voit une copie de tout le trafic échangé avec le routeur ; un PC sur Gi1/0/11 voit le trafic du VLAN Voix.

Point matériel important à présenter

Le premier switch utilisé (Catalyst 3550, IOS 12.1) ne supportait qu'une **seule** interface de destination et refusait la source « vlan » : impossible d'y faire deux sessions. Nous avons donc basculé sur un **Catalyst 3850**, plus récent, qui gère plusieurs sessions SPAN et la source VLAN — ce qui permet de répondre exactement au sujet.

Validation

show monitor session all affiche bien les 2 sessions (Session 1 : source Gi1/0/1 both → Gi1/0/10 ; Session 2 : source VLAN 100 → Gi1/0/11).

IV Interconnexion des réseaux (UHA, NAT, DNS)

But : Relier le routeur au réseau de l'UHA pour donner Internet à tous les VLANs, via un NAT de surcharge (PAT) basé sur une ACL nommée NAT, et un forwarder DNS pointant vers le résolveur de l'UHA (10.252.4.42).

1. Liaison vers l'UHA (port Gi0/1, adresse en DHCP)

```
configure terminal
interface Gi0/1
ip address dhcp
no shutdown
exit
```

L'UHA attribue une adresse publique au routeur (ex. 10.129.7.228) et fournit la route par défaut. Le port passe en up/up.

2. NAT de surcharge (PAT) basé sur l'ACL nommée NAT

```
! Interfaces internes (inside)
interface Gi0/0.100
ip nat inside
! ... idem .200 / .300 / .400

! Interface externe (outside)
interface Gi0/1
ip nat outside
exit

! ACL nommée NAT : les 4 sous-réseaux
ip access-list standard NAT
permit 10.252.6.0 0.0.0.63
permit 10.252.6.64 0.0.0.63
permit 10.252.6.128 0.0.0.63
permit 10.252.6.192 0.0.0.63
exit

! NAT surcharge : toutes les IP privées -> IP du port Gi0/1
ip nat inside source list NAT interface Gi0/1 overload

! Route par défaut + DNS forwarder
ip route 0.0.0.0 0.0.0.0 dhcp
ip name-server 10.252.4.42
```

Le NAT « overload » (PAT) traduit toutes les adresses privées des VLANs vers l'unique adresse publique du routeur, en distinguant les connexions par le numéro de port. C'est ce qui permet à tout le réseau de sortir sur Internet avec une seule IP UHA.

Validation

- ping 8.8.8.8 depuis le routeur : 100 % de réussite.
- Depuis un poste : ping 8.8.8.8 et ping google.fr fonctionnent (le second prouve aussi la résolution DNS via le forwarder).
- show ip nat translations montre les traductions actives.

V Services du serveur (Debian, VLAN 300)

But : Déployer sur le serveur Debian un service web (Apache2), un service FTP, un service Syslog pour centraliser les logs du routeur, et synchroniser l'horloge du routeur par NTP (UTC+2) sur fr.pool.ntp.org.

1. Service Web — Apache2

```
apt update
apt install apache2 -y
systemctl status apache2
```

Test : depuis un navigateur, <http://10.252.6.130> affiche la page Apache par défaut.

2. Service FTP — vsftpd

```
apt install vsftpd -y

# Correction du démarrage (erreur 500 OOPS: socket) :
# dans /etc/vsftpd.conf, forcer l'IPv4
# listen=YES
# listen_ipv6=NO

systemctl restart vsftpd
systemctl status vsftpd # -> active (running)
```

Par défaut vsftpd tentait d'écouter en IPv6 et plantait (« 500 OOPS: socket »). En forçant `listen=YES` / `listen_ipv6=NO` le service démarre correctement.

3. Service Syslog — centraliser les logs du routeur

Côté serveur Debian (rsyslog, réception réseau UDP/514) :

```
# Dans /etc/rsyslog.conf, décommenter :
module(load="imudp")
input(type="imudp" port="514")

systemctl restart rsyslog
```

Côté routeur (envoi des logs vers le serveur) :

```
configure terminal
logging host 10.252.6.130
logging trap informational
```

Validation : `tail -f /var/log/syslog` sur le serveur fait apparaître les messages envoyés par le routeur (source 10.252.6.129).

4. NTP — synchroniser l'horloge du routeur

```
! Sur le routeur
configure terminal
clock timezone CEST 2
ntp server fr.pool.ntp.org
```

Le fuseau est réglé sur UTC+2 (CEST) et le routeur se synchronise sur fr.pool.ntp.org (résolu via le forwarder DNS de l'UHA). La synchronisation NTP peut prendre quelques minutes (`show ntp status` / `show ntp associations`).

VI Sécurité du serveur (iptables)

But : Protéger le serveur Debian avec un pare-feu iptables : autoriser uniquement DNS, HTTP/HTTPS, le port 8080 et le FTP ; permettre au serveur de pinger mais bloquer les ping entrants ; tout bloquer par défaut.

Règles (serveur Debian)

```
# Politique par défaut : tout bloquer en entree
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT

# Reponses aux connexions sortantes + loopback
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -i lo -j ACCEPT

# DNS
iptables -A INPUT -p udp --dport 53 -j ACCEPT
iptables -A INPUT -p tcp --dport 53 -j ACCEPT

# HTTP / HTTPS / 8080
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -j ACCEPT
iptables -A INPUT -p tcp --dport 8080 -j ACCEPT

# FTP
iptables -A INPUT -p tcp --dport 21 -j ACCEPT
iptables -A INPUT -p tcp --dport 20 -j ACCEPT

# Ping : le serveur peut pinger, mais on bloque les ping entrants
iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
```

Logique de la règle ICMP (le point clé du sujet)

- Le serveur **envoie** un ping → autorisé par la politique OUTPUT ACCEPT.
 - La **réponse** revient → autorisée par la règle ESTABLISHED,RELATED.
 - Quelqu'un veut **pinger le serveur** (echo-request entrant) → bloqué par la règle DROP.
- Résultat : « le serveur peut faire des ping mais pas l'inverse », conforme au sujet.

Validation

- Depuis un autre poste : ping 10.252.6.130 → échoue (bloqué).
- Depuis le serveur : ping 8.8.8.8 → fonctionne.
- iptables -L -v -n liste les règles (chaîne INPUT en policy DROP).

À ne pas oublier

Les règles iptables sont perdues au redémarrage. Pour les rendre permanentes : apt install iptables-persistent -y puis netfilter-persistent save.

Récapitulatif — ce qui a été réalisé

Partie	Réalisé	Équipement
I — VLAN	4 VLANs, trunk 802.1Q, DHCP, ACL ping 200→400	Switch + Routeur
II — Protection	STP + BPDU Guard sur les ports d'accès	Switch

III — Mirroring	2 sessions SPAN (port routeur + VLAN 100)	Switch 3850
IV — Interco	Liaison UHA, NAT surcharge (ACL NAT), DNS, Internet	Routeur
V — Services	Apache2, FTP (vsftpd), Syslog, NTP	Serveur + Routeur
VI — Sécurité	iptables : services autorisés, ping unidirectionnel	Serveur